

TaxiMobile: architecture and delivery readiness

Technical investment in a cooperative, city-governed taxi platform

Engineering assessment of the current working tree

6 September 2026

84% estimated engineering scope implemented

Public launch / live pilot blocked; 0 of 19 P0 gates accepted

The engineering decision

Substantial implementation; incomplete operating evidence

The repository contains shared passenger/driver apps, applicant and operations web, authoritative ride/payment domains, city configuration and deployment tooling.

- ▶ 84% is a weighted source-scope estimate, not test coverage or time spent.
- ▶ Verified contacts, privacy self-service, payout accounting and governance controls still need work.
- ▶ Hosting, providers, devices, security and staffed city acceptance remain open.
- ▶ Next milestone: an immutable candidate and accepted staging, followed by bounded promotion.

Repository: [docs/readiness.md](#); [docs/gaps.md](#)

What the audit includes

Starting inventory

- ▶ 922 project files, including untracked source.
- ▶ 23 backend domain directories.
- ▶ 48 Alembic migrations; head 0048.
- ▶ 110 backend test files; 59 Kotlin test files.

Evidence boundary

- ▶ Structural and targeted critical-path review.
- ▶ Fresh unit/API and source-contract checks.
- ▶ Generated state and secrets excluded.
- ▶ No hosted, legal, bank, real-device or remote-CI acceptance inspected.

Candidate identity

Current dirty working tree over `f85417bcc8b432477fd88e87f5bd557a43898447`; that commit alone does not identify the tested implementation.

Product constraints shape the architecture

- ▶ Licensed taxi participation: an account is not a verified driver or city authorization.
- ▶ Driver agency: deterministic dispatch and explicit acceptance; no invented surge incentives.
- ▶ Supply privacy: no pre-assignment taxi markers, counts or candidate rankings.
- ▶ Transparent economics: versioned tariffs, named fee base, explicit funding mode.
- ▶ City rollout: independently configured and accepted operating jurisdictions.
- ▶ Backend authority: identity, eligibility, assignment, pricing and payment status.

Scope discipline

Card/CMI, general chat, direct emergency dispatch and speculative scale services must not be presented as launch capabilities.

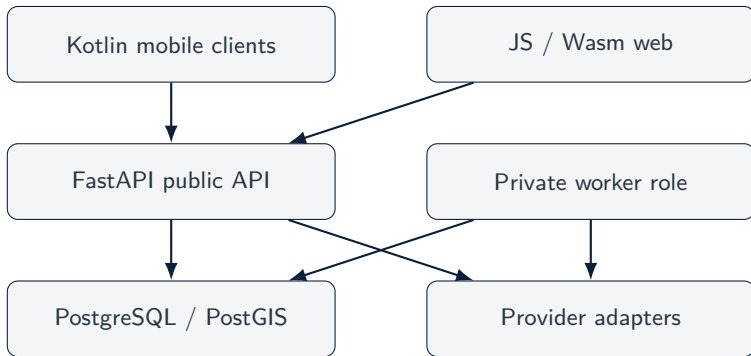
Repository: [docs/product.md](#); [docs/operations.md](#); [docs/payments.md](#)

Four experiences, distinct authority

Surface	Main workflow	Boundary
Passenger mobile	Estimate, request, track assigned ride, receipt, support	Own resources; no supply browsing.
Driver mobile	Apply, qualify, become available, accept, complete, earnings	Current account, vehicle, credential and city/service authority.
Applicant web	Recruiting city catalog, application, protected documents	Applicant-owned records; fail-closed upload dependencies.
Operations web	City configuration, review, money/cases, grants and audit	Active scoped staff permissions and operations MFA.

Repository: `TaxiMobile/shared/`; `TaxiMobile/webApp/src/webMain/`

Runtime topology



Source of truth

Database transactions govern business state. Client caches and notification payloads cannot acknowledge money or allocate a driver.

Modular monolith and shared clients

- ▶ Python FastAPI + SQLAlchemy/asyncpg + Alembic; PostGIS geographic authority.
- ▶ API and worker are separate roles from the same backend image.
- ▶ Transactional outbox and PostgreSQL fanout avoid premature queue/cache services.
- ▶ Kotlin separates gateway/DTO, domain, coordinator, UI and native adapters.
- ▶ Android passenger/driver flavors and iOS schemes share behavior without merging roles.
- ▶ Browser applicant/operations roots are distinct from passenger/driver mobile composition.

Repository: [docs/architecture.md](#); [docs/implementation.md](#); [backend/src/taximobile_api/](#)

HTTP contract and unknown command outcomes

- ▶ Stable `/api/v1` router; runtime OpenAPI describes mounted operations.
- ▶ Fresh contract validation: 82 mobile HTTP + 1 WebSocket; 98 web HTTP operations.
- ▶ The backend validates identity, scope, state, versions and required idempotency.
- ▶ A timeout can occur after commit; it is not necessarily a rejected command.
- ▶ Clients reload authoritative state and retain actionable errors; they do not automatically replay mutations.

What route matching does not prove

Payload compatibility, all authorization branches, full browser journeys and provider/device delivery require additional tests.

Repository: `infra/scripts/validate_mobile_api_contract.py`; `validate_web_api_contract.py`

Identity and scoped staff security

- ▶ Argon2 login, constrained JWT claims and current server-side session checks.
- ▶ Revocation and suspension invalidate authority beyond token signature validity.
- ▶ Operations uses separate sessions, scoped grants, MFA and recent-MFA boundaries.
- ▶ Secure cookie/CSRF implementation exists; production acceptance remains separate.
- ▶ Staff create/revoke UI now exists with typed confirmation and non-replay after MFA.

Still incomplete

Independent approval, authoritative staff lifecycle and last-admin continuity; verified contact ownership and production recovery exercises.

Repository: [domains/auth/](#); [domains/administration/](#); GAP-005/020/022

Account recovery is explicit and bounded

- ▶ Eight independent 100-bit offline codes after password reauthentication.
- ▶ Domain-separated digests; 180-day expiry; one-time plaintext display for user custody.
- ▶ Recovery consumes the complete set and revokes sessions/push in one transaction.
- ▶ Staff cannot use the ordinary mobile recovery route.
- ▶ Mobile saved-code acknowledgement, session listing/revocation and password change exist.

Not yet delivered

Verified email/phone ownership, delivery-based reset, self-service account deletion and personal-data export. Offline codes do not establish verified contacts.

Repository: [docs/auth.md](#); [docs/security.md](#); GAP-020/021

Ride and offer state are different aggregates

Successful live ride sequence

REQUESTED → MATCHING → ACCEPTED → DRIVER_EN_ROUTE → DRIVER_ARRIVED → IN_PROGRESS → COMPLETED

- ▶ Offers separately track pending, accepted, declined, expired and cancelled states.
- ▶ No “OFFERED” value is added to the implemented RideStatus enum.
- ▶ Cancellation is restricted to documented pre-trip states; ordinary cancellation cannot erase an in-progress ride.
- ▶ Completion finalizes fare and creates a pending payment; collection is separate.
- ▶ Participant reads and coordination depend on backend-confirmed assignment.

Repository: `domains/rides/models.py`; `domains/rides/service.py`

Dispatch: discover, rank, lock, revalidate

1. Discover a bounded geographic set satisfying account, vehicle, credential, city, service, availability and location conditions.
2. Rank deterministically using proximity, idle time and recent assignment signals.
3. Lock one candidate with skip-locked semantics rather than reserve all supply.
4. Recheck global account authority and eligibility after the lock.
5. Create the offer, or try another candidate / retry within the matching deadline.

Contention is not absence

A temporarily locked candidate must not make the system falsely conclude that the city has no eligible supply.

Repository: `domains/matching/service.py`; dispatch-contention integration cases

One active live ride per driver

$$\forall d : |\{r : r.driver_id = d \wedge r.status \in A\}| \leq 1$$

A contains ACCEPTED, DRIVER_EN_ROUTE, DRIVER_ARRIVED and IN_PROGRESS.

- ▶ Migration 0048 creates a partial unique index on assigned drivers in active states.
- ▶ It freezes ride writes and refuses existing conflicts instead of repairing history silently.
- ▶ Service locks serialize supported commands; the index also rejects bypassing writes.
- ▶ Immediate, scheduled and mixed-origin assignments must respect the same invariant.

Acceptance gap

Local race tests are not representative multi-replica city-load evidence.

Published routes and scheduled bookings

Fixed routes

- ▶ Versioned direction, endpoints, stops, geometry and flat fare.
- ▶ Publication is separate from driver supply.
- ▶ Catalog visibility never promises a taxi.
- ▶ Offers still require service authorization and acceptance.

Scheduling

- ▶ Reservation is separate from the live ride.
- ▶ Policy defines horizon, opening, surcharge and cancellation.
- ▶ Commitment/handoff recheck authority and location.
- ▶ Rollback/replay must not duplicate live rides.

Repository: `domains/fixed_routes/`; `domains/scheduled_bookings/`; scheduling integration tests

Exact and conserved financial components

Let T be transport fare, $S = S_d + S_o$ scheduling surcharge and F operator fee. Let $a = 1$ for passenger-funded fee, $a = 0$ for driver deduction.

$P = T + S + aF$	passenger total
$D = T + S_d - (1 - a)F$	driver net
$O = F + S_o$	operator allocation
$P = D + O$	conservation

- ▶ Decimal arithmetic; half-up rounding to 0.01; defined transport-fare percentage base.
- ▶ Currency, policy shape and driver-net floor validated server-side.
- ▶ Policy versions and amounts snapshotted; later edits do not rewrite history.

Repository: `domains/pricing/service.py::calculate_financial_quote`

Money example: funding mode changes the receipt

Illustrative arithmetic, not an approved tariff

$T = 100.00$, scheduling surcharge $S = 0$, operator fee $F = 10.00$.

Funding mode	Passenger	Driver net	Operator
Driver deduction	100.00	90.00	10.00
Passenger surcharge	110.00	100.00	10.00

- ▶ Both conserve passenger total into driver and operator allocations.
- ▶ Client estimates must not invent a fee base or hide the deduction.
- ▶ An earning allocation is not proof of a payout or a bank transfer.

Repository: [docs/pricing.md](#); [docs/payments.md](#)

Payment settlement requires a separate authority

- ▶ Ride completion creates one pending payment.
- ▶ Cash: authorized driver confirms collection; receipt initially remains pending.
- ▶ Manual transfer: passenger claims; scoped staff reconciles exact external evidence.
- ▶ Processing means awaiting review, never “paid”.
- ▶ Non-legacy cities resolve active versioned capabilities and recipient instructions from the exact city bundle.
- ▶ Card/CMI is deferred; reserved enum values are not advertised payment options.

Repository: [domains/payments/](#); versioned-payment and cash-workload tests

Refunds exist; payout accounting does not

Implemented refunds

- ▶ Append-only, completed-payment linkage.
- ▶ Exact positive amount and bounded cumulative total.
- ▶ Locking, idempotency, reason, evidence and audit.
- ▶ Full operator funding; zero driver recovery.

GAP-024

- ▶ Settlement periods and payable balance.
- ▶ Cash offsets and payout instructions.
- ▶ Remittance/reconciliation ledger.
- ▶ Accountant-approved correction and dispute policy.

Decision dependency

Approve accounting policy before introducing payout schema or silently reusing earnings as a ledger.

Durable effects and refresh-only notifications

- ▶ Transactional outbox keeps delivery work associated with committed business state.
- ▶ PostgreSQL fanout serves multiple connected API instances.
- ▶ WebSocket and FCM/APNs hints trigger authorized reads.
- ▶ Invalid registrations, bounded retries, dead letters and reviewed replay are explicit paths.
- ▶ Payloads and logs must avoid exact location, transfer instructions and safety narratives.

Failure interpretation

No notification does not mean no commit. A delivered notification is not independent authority for assignment or settlement.

Repository: `domains/outbox/`; `core/live_events.py`; `integrations/push/`

Eight background loops need observable progress

Loop	Operational responsibility
Matching	Dispatch progression and expiry/retry.
Outbox	Durable delivery and failure recovery.
Credentials	Eligibility lifecycle.
Scheduling	Due reservations and live handoff.
Analytics	Aggregate projection refresh.
Case alerts	Overdue support/safety escalation.
Case retention	Eligible minimization with hold protection.
Document retention	Protected-file lifecycle through configured storage.
Readiness requires supervisor health and positive loop progress; liveness alone is insufficient.	

Repository: `workers/application.py`; production readiness and monitoring validators

Maps and locations: source versus service quality

- ▶ MapLibre native composition, neutral fallback and shared route/point overlays.
- ▶ Valhalla/GraphHopper adapters behind a selected backend routing contract.
- ▶ Authenticated normalized place search/reverse lookup with PostGIS pickup authority.
- ▶ One-shot location plus foreground-only online driver heartbeat are implemented.
- ▶ Continuous background tracking is not implied by that heartbeat.

Open acceptance

Real graph/style/geocoder, data/license and query privacy, multilingual search, device battery/network behavior and accepted Arabic routing narration.

Repository: [GAP-006/007/008/026](#); native location and MapLibre source sets

Mobile and browser quality are separate test boundaries

- ▶ Shared coordinators improve consistency; native lifecycle and permissions still differ.
- ▶ English/French/Arabic catalog parity does not prove accessibility or RTL layout.
- ▶ Backend-confirmed success drives UI feedback; unknown outcomes retain recovery paths.
- ▶ JS/Wasm component tests do not establish complete hosted applicant/staff journeys.
- ▶ Windows JVM verification cannot execute iOS simulator tests.
- ▶ Signed release, provider config, R8/dSYM and controlled crash evidence remain gates.

Repository: [docs/design.md](#); [docs/ui.md](#); [docs/extended_ui.md](#); GAP-013/014/025

City configuration is a coherent versioned bundle

- ▶ Market → city/operator assignment → enabled service.
- ▶ Service area, tariff, fee, scheduling and payment references must be compatible.
- ▶ Fixed-route publication and driver authorization stay service/city scoped.
- ▶ Operations UI resolves backend versions rather than constructing authority in the browser.
- ▶ Readiness, pause/resume and closeout controls exist in source.

Governance limit

Valid evidence records do not prove that the uploaded legal evidence is true, the recruited cohort is eligible or the duty roster will answer.

Repository: docs/operations.md; domains/markets/; operations web control plane

Documents, support and safety need real operators

- ▶ Protected application PDF/JPEG/PNG workflow fails closed without storage/scanning.
- ▶ Scoped reviewer access, encrypted document boundary and retention code exist.
- ▶ Support and restricted safety records have separate visibility and triage.
- ▶ Overdue paging, acknowledgement, legal holds and minimization are implemented.
- ▶ Platform safety reporting does not place emergency calls or dispatch help.

Acceptance includes people and every store

Real scanner/key operation, trained shifts, pager drills, incident response and backup/log/provider expiry must be demonstrated.

Repository: GAP-011/012/017/030/031

Production manifests are not deployed infrastructure

- ▶ Separate API/worker roles, restricted operations ports and readiness boundaries.
- ▶ Bounded JSON logs and authenticated aggregate metrics.
- ▶ Prometheus, Alertmanager, Loki, Alloy and Grafana overlay/validators.
- ▶ Migration/backup/restore and deployment-verification scripts.
- ▶ CI provenance, contract checks and image supply-chain definitions.

Not established by source files

Accepted host/region, TLS/DNS, private networks, secret rotation, immutable registry, managed recovery, real monitoring targets, pager ownership or HA.

Repository: `infra/deploy/`; `infra/scripts/`; `.github/workflows/ci.yml`

Restore and migration must match the candidate

- ▶ Current schema head: 20260903_0048, 48 ordered migrations.
- ▶ Offline SQL generation is not a live lock/permissions/historical-data rehearsal.
- ▶ Older documented restore at 0029 remains historical, not a current-head result.
- ▶ Required: isolated 0048 restore, integrity/readiness checks and measured recovery.
- ▶ RPO/RTO, retention, encryption and legal-hold implications need approved ownership.
- ▶ Forward-fix/rollback must respect schema compatibility and irreversible data changes.

Repository: GAP-003; docs/database.md; infra recovery scripts

Fresh checks and historical evidence

Check	Result / qualification
Backend unit/API	699 passed; 173 deprecation warnings; 45.52 seconds.
Infrastructure/mobile script tests	11 + 30 passed.
Mobile/backend contract	82 HTTP + one WebSocket passed.
Web/backend contract	98 HTTP operations passed.
Shared JVM forced rerun	178 passed; zero failures/errors/skips.
Full fresh-PostGIS suite	Existing record: 819 passed; not rerun for this documentation assessment.
Native/hosted/remote CI acceptance	No fresh acceptance claimed.

Repository: [docs/readiness.md](#) for commands, scope and final verification

How the 84 percent estimate is calculated

Explicit source-scope rubric

0 requirement only; 1 scaffold; 2 material implementation with a major missing workflow; 3 main path/tests with meaningful remaining scope; 4 defined core source path and meaningful boundary tests.

$$C = \sum_i w_i (s_i/4) = 336/4 = 84\%, \quad \sum_i w_i = 100.$$

- ▶ Weights are assessment judgments, not approved budgets or elapsed effort.
- ▶ Approximately 75–90% sensitivity range; not a statistical confidence interval.
- ▶ Deferred products excluded; identified launch engineering gaps included.
- ▶ P0 acceptance is a veto, not another weighted score: 0/19 accepted.

Implementation matrix: core domains

Workstream	Weight	Rating / 4	Contribution
Identity/recovery	8	3	6.00
Drivers/recruitment	8	4	8.00
Immediate rides/coordination	10	4	10.00
Matching/integrity	8	4	8.00
Pricing/economics	7	4	7.00
Payments/settlement	8	3	6.00
Fixed routes/scheduling	7	4	7.00

Missing verified contact and payout paths cap their workstreams below four. Four indicates source/tests, never field or financial acceptance.

Repository: docs/readiness.md: evidence and missing scope per row

Implementation matrix: clients and enabling systems

Workstream	Weight	Rating / 4	Contribution
Mobile	10	3	7.50
Web	7	3	5.25
Maps/places/navigation/delivery	6	3	4.50
Support/safety	5	3	3.75
Control plane/governance	5	3	3.75
Privacy rights/lifecycle	4	2	2.00
Analytics	2	3	1.50
Deployment/observability	3	3	2.25
CI/supply chain/release	2	3	1.50

Repository: Combined weights = 100; combined contribution = 84.00

Engineering gaps to resolve before claiming completion

1. Verified contacts and production recovery: choose/approve identity and delivery behavior.
2. Privacy self-service: deletion/export with legal holds and cross-store consequences.
3. Staff governance: independent approval and last-admin continuity.
4. Settlement accounting: periods, balances, cash offsets, remittance and reconciliation.
5. Complete client/provider integration: narration, device/browser journeys and failure behavior.

Policy before schema

The audit records these dependencies; it does not invent accounting, legal, recovery or major authorization rules to accelerate a percentage.

Repository: GAP-020-024; GAP-025/026/030

Evidence-based delivery sequence

1. Review uncommitted work; identify a clean candidate and immutable artifacts.
2. Resolve policy-dependent software scope in owning domain documents.
3. Accept isolated staging, current-head restore and selected providers.
4. Complete device/browser, independent security and defined capacity gates.
5. Train scoped operators and rehearse money, safety, paging and recovery.
6. Promote to bounded field validation, pilot and public city only with explicit acceptance.

Repository: [docs/workflow.md](#); [docs/testing.md](#) T0–T10; [docs/gaps.md](#)

A task is one complete behavior slice

- ▶ Start: requirement/GAP, actor/scope, trigger, outcome and rejected cases.
- ▶ Trace existing architecture and implementation before adding code.
- ▶ Define transaction, schema/API impact and meaningful negative/failure tests.
- ▶ Connect shared gateway/coordinator and native adapter with honest UI state.
- ▶ Update owning docs; run checks at the changed boundary.
- ▶ Handoff: source/artifact identity, commands, observed results, limits and open gates.

Conflict rule

Pause the affected implementation if normative requirements conflict with code; do not rewrite policy to approve a convenient implementation.

Promotion needs traceable evidence

Stage	Necessary evidence boundary
T0–T2	Source/static, unit/component and simulated-persona contracts.
T3	Isolated migrated PostGIS and multi-role system transactions.
T4	Browser, emulator and physical-device lab.
T5	Candidate staging: restore, providers, security and realistic load/failure.
T6	Trained staff, synthetic cases, money and incident rehearsal.
T7–T8	Approved bounded field validation then real-user pilot.
T9–T10	Public city acceptance, closeout and proven repeatable expansion.
Record owner, date, candidate/artifacts, schema, city/provider versions, scenario, observed result, evidence location, expiry and decision.	

Documentation now has a clearer division of responsibility

- ▶ `readiness.md`: dated percentage, matrix, findings and fresh/historical results.
- ▶ `workflow.md`: task contracts, commands, handoffs and promotion sequence.
- ▶ `gaps.md`: detailed open-work closure and acceptance criteria.
- ▶ `testing.md`: T0–T10 scenarios; workload companion for guarded HTTP experiments.
- ▶ Domain docs retain product, API, schema, security and UI authority.
- ▶ TeX/PDF reports communicate the assessment and must track the same evidence date.

Repository: `docs/README.md` is the entry point

Decision requested from the project owner

Prioritize an accepted operating system around the source

The next work is candidate consolidation, policy-dependent completion and staged acceptance for one bounded city, with explicit owners and measurable stop criteria.

- ▶ Keep the implemented modular architecture and backend authority boundaries.
- ▶ Resolve settlement/privacy/identity/governance decisions before coding those gaps.
- ▶ Identify infrastructure, security, provider, city, payment and safety owners.
- ▶ Require evidence at each promotion gate; do not equate 84% source scope with permission to launch.

Repository: Communication artifact only; no deployment or policy approval is implied.